0. SETUP
1. TRANSFORMER MODELS
2. USING 🤗 TRANSFORMERS
Introduction
Behind the pipeline
Models
Tokenizers
Handling multiple sequences
Putting it all together
Basic usage completed!
End-of-chapter quiz
3. FINE-TUNING A PRETRAINED MODEL
4. SHARING MODELS AND TOKENIZERS
5. THE 🤗 DATASETS LIBRARY
6. THE 🤗 TOKENIZERS LIBRARY
7. MAIN NLP TASKS
8. HOW TO ASK FOR HELP
9. BUILDING AND SHARING DEMOS NEW
COURSE EVENTS

Join the Hugging Face community
and get access to the augmented documentation experience
Collaborate on models, datasets and Spaces
Faster examples with accelerated inference
Switch between documentation themes
Sign Up to get started

Handling multiple sequences
Models expect a batch of inputs
Padding the inputs
Attention masks
Longer sequences

Pytorch TensorFlow

Ask a question Open in Colab Open In Studio Lab

Handling multiple sequences



In the previous section, we explored the simplest of use cases: doing inference on a single sequence of a small length. However, some questions emerge already:

- How do we handle multiple sequences?
- How do we handle multiple sequences of *different lengths*?
- Are vocabulary indices the only inputs that allow a model to work well?
- Is there such a thing as too long a sequence?

Let's see what kinds of problems these questions pose, and how we can solve them using the 🤗 Transformers API.

Models expect a batch of inputs

In the previous exercise you saw how sequences get translated into lists of numbers. Let's convert this list of numbers to a tensor and send it to the model:

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)

sequence = "I've been waiting for a HuggingFace course my whole life."

tokens = tokenizer.tokenize(sequence)
ids = tokenizer.convert_tokens_to_ids(tokens)
input_ids = torch.tensor(ids)
# This line will fail.
model(input_ids)
```

```
IndexError: Dimension out of range (expected to be in range of [-1, 0], but got 1)
```

Oh no! Why did this fail? "We followed the steps from the pipeline in section 2.

The problem is that we sent a single sequence to the model, whereas 🤗 Transformers models expect multiple sentences by default. Here we tried to do everything the tokenizer did behind the scenes when we applied it to a `sequence`. But if you look closely, you'll see that the tokenizer didn't just convert the list of input IDs into a tensor, it added a dimension on top of it:

```
tokenized_inputs = tokenizer(sequence, return_tensors="pt")
print(tokenized_inputs["input_ids"])
```

```
tensor([[  101,  1045,  1005,  2310,  2042,  3403,  2005,  1037, 17662, 12172,
          2607,  2026,  2878,  2166,  1012,   102]])
```

Let's try again and add a new dimension:

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)

sequence = "I've been waiting for a HuggingFace course my whole life."

tokens = tokenizer.tokenize(sequence)
ids = tokenizer.convert_tokens_to_ids(tokens)

input_ids = torch.tensor([ids])
print("Input IDs:", input_ids)

output = model(input_ids)
print("Logits:", output.logits)
```

We print the input IDs as well as the resulting logits — here's the output:

```
Input IDs: [[ 1045,  1005,  2310,  2042,  3403,  2005,  1037, 17662, 12172,  2607, 2026,  2878,  2166,  1
Logits: [[-2.7276,  2.8789]]
```

Batching is the act of sending multiple sentences through the model, all at once. If you only have one sentence, you can just build a batch with a single sequence:

```
batched_ids = [ids, ids]
```

This is a batch of two identical sequences!

> ✏️ **Try it out!** Convert this `batched_ids` list into a tensor and pass it through your model. Check that you obtain the same logits as before (but twice)!

Batching allows the model to work when you feed it multiple sentences. Using multiple sequences is just as simple as building a batch with a single sequence. There's a second issue, though. When you're trying to batch together two (or more) sentences, they might be of different lengths. If you've ever worked with tensors before, you know that they need to be of rectangular shape, so you won't be able to convert the list of input IDs into a tensor directly. To work around this problem, we usually *pad* the inputs.

Padding the inputs

The following list of lists cannot be converted to a tensor:

```
batched_ids = [
    [200, 200, 200],
    [200, 200]
]
```

In order to work around this, we'll use *padding* to make our tensors have a rectangular shape. Padding makes sure all our sentences have the same length by adding a special word called the *padding token* to the sentences with fewer values. For example, if you have 10 sentences with 10 words and 1 sentence with 20 words, padding will ensure all the sentences have 20 words. In our example, the resulting tensor looks like this:

```
padding_id = 100

batched_ids = [
    [200, 200, 200],
    [200, 200, padding_id],
]
```

The padding token ID can be found in `tokenizer.pad_token_id`. Let's use it and send our two sentences through the model individually and batched together:

```
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)

sequence1_ids = [[200, 200, 200]]
sequence2_ids = [[200, 200]]
batched_ids = [
    [200, 200, 200],
    [200, 200, tokenizer.pad_token_id],
]

print(model(torch.tensor(sequence1_ids)).logits)
print(model(torch.tensor(sequence2_ids)).logits)
print(model(torch.tensor(batched_ids)).logits)
```

```
tensor([[ 1.5694, -1.3895]], grad_fn=<AddmmBackward>)
tensor([[ 0.5803, -0.4125]], grad_fn=<AddmmBackward>)
tensor([[ 1.5694, -1.3895],
        [ 1.3373, -1.2163]], grad_fn=<AddmmBackward>)
```

There's something wrong with the logits in our batched predictions: the second row should be the same as the logits for the second sentence, but we've got completely different values!

This is because the key feature of Transformer models is attention layers that *contextualize* each token. These will take into account the padding tokens since they attend to all of the tokens of a sequence. To get the same result when passing individual sentences of different lengths through the model or when passing a batch with the same sentences and padding applied, we need to tell those attention layers to ignore the padding tokens. This is done by using an attention mask.

Attention masks

Attention masks are tensors with the exact same shape as the input IDs tensor, filled with 0s and 1s: 1s indicate the corresponding tokens should be attended to, and 0s indicate the corresponding tokens should not be attended to (i.e., they should be ignored by the attention layers of the model).

Let's complete the previous example with an attention mask:

```
batched_ids = [
    [200, 200, 200],
    [200, 200, tokenizer.pad_token_id],
]

attention_mask = [
    [1, 1, 1],
    [1, 1, 0],
]

outputs = model(torch.tensor(batched_ids), attention_mask=torch.tensor(attention_mask))
print(outputs.logits)
```

```
tensor([[ 1.5694, -1.3895],
        [ 0.5803, -0.4125]], grad_fn=<AddmmBackward>)
```

Now we get the same logits for the second sentence in the batch.

Notice how the last value of the second sequence is a padding ID, which is a 0 value in the attention mask.

> ✏️ **Try it out!** Apply the tokenization manually on the two sentences used in section 2 ("I've been waiting for a HuggingFace course my whole life." and "I hate this so much"). Pass them through the model and check that you get the same logits as in section 2. Now batch them together using the padding token, then create the proper attention mask. Check that you obtain the same results when going through the model!

Longer sequences

With Transformer models, there is a limit to the lengths of the sequences we can pass the models. Most models handle sequences of up to 512 or 1024 tokens, and will crash when asked to process longer sequences. There are two solutions to this problem:

- Use a model with a longer supported sequence length.
- Truncate your sequences.

Models have different supported sequence lengths, and some specialize in handling very long sequences. Longformer is one example, and another is LED. If you're working on a task that requires very long sequences, we recommend you take a look at those models.

Otherwise, we recommend you truncate your sequences by specifying the `max_sequence_length` parameter:

```
sequence = sequence[:max_sequence_length]
```